6시간 C 언어 다음 단계 학습서

문법 이해 다음에 해야 할 것: 구현, 디버깅, 자료구조 연결

대상: C 기본 문법을 한 번 끝까지 보고, 이제 이해했다에서 직접 구현할 수 있다로 넘어가려는 재혁님.

이 PDF의 목표는 문법 암기가 아니라, 현재 저장소 문제 파일로 자연스럽게 넘어갈 수 있는 학습 흐름을 만드는 것입니다.

핵심 목표

- 배운 문법을 실제 문제 코드에 연결한다.
- 배열, 포인터, 구조체, 함수가 같이 쓰이는 상황을 익힌다.
- Linked List, Stack/Queue, Tree 문제를 보고 겁먹지 않게 만든다.
- 컴파일 에러와 런타임 오류를 스스로 좁혀가는 습관을 만든다.

6시간 전체 흐름

- 1시간차: 문법을 짧은 구현으로 다시 묶기
- 2시간차: 포인터와 구조체 체감하기
- 3시간차: Linked List 손으로 만들기
- 4시간차: Stack / Queue 규칙 익히기
- 5시간차: Tree와 재귀 구조 읽기
- 6시간차: 현재 저장소 문제 한 개 완주하기

성공 기준

모르는 것이 하나도 없는 상태가 아니라, 막혀도 어디서부터 확인해야 하는지 아는 상태면 성공입니다.

1시간차: 문법을 구현으로 다시 묶기

변수, 함수, 배열, 구조체를 각각 따로 외우지 말고 작은 예제로 다시 연결합니다. 각 항목당 10분 정도로 짧게 돌리는 것이 핵심입니다.

연습 1. 함수와 반환값

```
int add(int a, int b) {\n    return a + b;\n}
```

- 함수 선언과 호출을 직접 세 번 써본다.
- return이 없으면 왜 문제가 되는지 말로 설명해본다.
- 입력 2개, 반환 1개 구조를 익힌다.

연습 2. 배열과 반복문

```
int arr[5] = {1, 2, 3, 4, 5};\nfor (int i = 0; i < 5; i++) {\n    printf(\
```

- i++, i += 2를 각각 바꿔 써본다.
- 배열 길이와 인덱스 차이를 구분한다.
- 오프바이원 실수를 일부러 만들고 고친다.

연습 3. 구조체

```
struct Student {\n    int id;\n    int score;\n};
```

체크 질문: 배열은 무엇을 저장하고, 포인터는 무엇을 저장하고, 구조체는 무엇을 묶는가?

2시간차: 포인터와 구조체를 진짜로 이해하기

자료구조는 결국 주소를 저장하고 따라가는 기술입니다. 포인터를 넘기지 못하면 Linked List도 바로 막힙니다.

```
#include <stdio.h>\n\nint main(void) {\n    int x = 10;\n    int* p = &x;\n\n    printf(\
```

여기서 꼭 느껴야 하는 것

- p는 주소이고, *p는 그 주소가 가리키는 실제 값이다.
- 값을 직접 건드리지 않아도 포인터를 통해 바꿀 수 있다.
- NULL 체크 없이 접근하면 위험하다.

구조체 포인터도 같은 원리

```
struct Student s;\nstruct Student* ps = &s;\nps->id = 1;\nps->score = 95;
```

목표: 주소를 따라가서 값을 바꾼다는 문장을 코드 없이 말로 설명할 수 있으면 충분합니다.

3시간차: Linked List를 손으로 만들기

배열처럼 옆 칸으로 가는 것이 아니라, 다음 노드의 주소를 따라간다는 감각만 정확히 잡으면 됩니다.

```
struct Node {\n    int data;\n    struct Node* next;\n};\n\nhead->data = 10;\nhead->next = second;\nsecond->data =
20;\nsecond->next = third;\nthird->data = 30;\nthird->next = NULL;
```

이 시간의 연습 순서

- 노드 3개 연결하기
- while (current != NULL) 로 순회하기
- 맨 앞에 노드 1개 추가하기
- 맨 끝에 노드 1개 추가하기

반드시 설명할 수 있어야 하는 문장

- Linked List는 각 노드가 다음 노드 주소를 저장하는 구조다.
- head는 첫 노드를 가리킨다.
- current = current->next 는 다음 노드로 이동하는 코드다.

추천 연결: 현재 저장소의 Linked List 첫 문제를 열고, 함수가 어떤 포인터를 받고 어떤 포인터를 바꾸는지만 먼저 읽어보세요.

4시간차: Stack / Queue는 규칙의 차이

새로운 마법처럼 보이지만 실제로는 넣는 규칙과 빼는 규칙이 다를 뿐입니다. Stack은 LIFO, Queue는 FIFO입니다.

```
int stack[100];\nint top = -1;\n\nvoid push(int x) {\n    top++;\n    stack[top] = x;\n}\n\nint pop(void) {\n    int value
= stack[top];\n    top--;\n    return value;\n}
```

이 시간 목표

- top, front, rear가 각각 무엇을 뜻하는지 구분한다.
- push/pop, enqueue/dequeue를 손으로 추적한다.
- 배열 버전과 연결 리스트 버전 차이를 말로 설명한다.

권장 미션

- 숫자 3개를 push 하고 pop 결과를 예상해본다.
- 숫자 3개를 enqueue 하고 dequeue 결과를 예상해본다.
- 비어 있을 때 pop 하면 어떻게 처리할지 직접 정한다.

5시간차: Tree와 재귀를 두려워하지 않기

Tree는 노드 하나가 자식 포인터를 여러 개 가질 수 있는 구조입니다. 재귀를 완벽히 외우는 것보다 구조를 읽는 것이 먼저입니다.

```
struct TreeNode {\n    int data;\n    struct TreeNode* left;\n    struct TreeNode* right;\n};\n\nvoid preorder(struct
TreeNode* node) {\n    if (node == NULL) return;\n    printf(\
```

재귀를 볼 때 확인할 것

- 이 함수는 언제 멈추는가?
- 현재 노드에서 무엇을 처리하는가?
- 다음에는 왼쪽과 오른쪽 중 어디로 가는가?

재혁님 단계의 핵심

- Linked List의 next 하나가 Tree에서는 left, right 두 개로 늘어난 느낌으로 이해한다.
- 루트, 왼쪽 자식, 오른쪽 자식을 그림으로 그릴 수 있어야 한다.
- 멈추는 조건과 다음 호출 방향 두 가지만 놓치지 않는다.

6시간차: 현재 저장소 문제 한 개 끝까지 가기

작은 예제에서 끝내지 말고 실제 문제 파일로 연결합니다. 한 문제를 완벽히 맞히는 것보다 코드 흐름을 읽고 한 칸씩 수정하는 힘이 중요합니다.

추천 진입 순서

- Linked List 첫 문제 파일 열기
- 입출력보다 함수 역할부터 읽기
- 주어진 틀에서 비어 있는 부분만 채우기
- 작은 입력으로 직접 테스트하기
- 출력이 틀리면 중간 값을 print 해서 좁히기

막혔을 때 순서

- 컴파일 에러를 한 개씩 먼저 없앤다.
- 출력값이 틀리면 중간 값을 찍어본다.
- 포인터면 주소와 값 둘 다 확인한다.
- 마지막 노드, 빈 구조, 경계 조건을 다시 본다.

오늘 끝나고 남겨야 할 것

- 내가 헷갈린 문법 3개
- 직접 구현한 예제 2개
- 스스로 설명 가능해진 개념 1개

다음 추천 순서: Linked List 한 문제 완주 -> Stack/Queue 한 문제 완주 -> Tree 코드 읽기.